Text-Based Prediction of Book Review Popularity

Team members:

Pallavi K G PES1PG25CS046

Pasupuleti Pavani PES1PG25CS047

Github link: https://github.com/pavanipasupuleti/Book-Review-Popularity

1. Problem Statement

Online platforms like Goodreads contain millions of user reviews, but only a small fraction of these reviews receive high engagement in the form of votes and comments.
The problem addressed in this project is to predict whether a book review will be popular or not based on its text content and engagement-related features.

Specifically, the task is formulated as a binary classification problem, where:

- Popular (1) → Review receives high engagement
- Not Popular (0) → Review receives low engagement

The objective is to build machine learning models that can automatically classify reviews as popular or not, helping platforms highlight impactful reviews.

2. Approach Used to Solve the Problem

The solution follows a machine learning pipeline consisting of the following steps:

1. Data Preparation
    - Goodreads review data is collected and stored in CSV format.
    - Engagement features such as number of votes and comments are used to define popularity.
2. Feature Engineering
    - Text features:
        - Number of words in the review
        - Average word length

        - Sentiment score using VADER
    - TF-IDF features extracted from review text to capture important words.
3. Label Generation
    - Reviews are labeled as popular or not popular based on total engagement using a predefined threshold.

4. Model Training
    - Multiple machine learning models are trained:
        - Logistic Regression
        - XGBoost
        - Neural Network (MLP)
5. Evaluation
    - Models are evaluated using:
        - Accuracy
        - Sensitivity (Recall)
        - Specificity
        - AUC score
6. Prediction
    - A trained model is saved and later used to predict popularity for new user-entered reviews.

3. Brief Overview of the Implementation

The implementation is modular and divided into multiple Python files:

- prepare_real_data.py
  Extracts raw Goodreads JSON data and converts it into a structured CSV file.
- preprocess_data.py
  Adds popularity labels and text-based features, producing a cleaned dataset for training.
- features.py
  Contains all feature engineering logic including sentiment analysis and TF-IDF vectorization.

- train.py
  Trains Logistic Regression, XGBoost, and Neural Network models and compares their performance.
- train_and_save_model.py
  Trains the final Logistic Regression model and saves:
  - Trained model
  - TF-IDF vectorizer
  - Feature scaler
- evaluate.py

  Computes evaluation metrics such as accuracy, sensitivity, specificity, and AUC.
- run_experiments.py

  Runs experiments on different feature subsets and saves results for comparison.
- predict_review.py

  Takes a new review as input and predicts whether it will be POPULAR or NOT POPULAR.
- run_logistic.py

  Logistic Regression experiment with feature selection and hyperparameter tuning using GridSearchCV, then evaluates the best model using test-set performance metrics.
- run_xgboost.py

  XGBoost classifies using selected feature subsets and evaluates its performance on the test data using standard classification metrics.
- run_neural_net.py

  simple neural network (MLP) with one hidden layer on numeric features and evaluates its performance using test-set classification metrics.

The project follows a clean pipeline where data flows from preprocessing → training → evaluation → prediction.

4. Results

```
=== Experiment Results  ===
Feature Subset      Model Under Sample     Hyperparameters  Accuracy  Sensitivity  Specificity      AUC
             A   Logistic          Yes               C=0.1  0.714829     0.107986     0.973457 0.681638
             B   Logistic          Yes               C=0.1  0.714829     0.107986     0.973457 0.681638
             C   Logistic          Yes               C=0.1  0.714829     0.107986     0.973457 0.681638
             A    XGBoost           No  trees=300, depth=4  0.708425     0.150008     0.946415 0.647083
             B    XGBoost           No  trees=300, depth=4  0.708425     0.150008     0.946415 0.647083
             C    XGBoost           No  trees=300, depth=4  0.708425     0.150008     0.946415 0.647083
             A Neural Net           No 1 hidden layer (32)  0.686412     0.354596     0.827827 0.646503
['review_id', 'book_id', 'user_id', 'rating', 'review_text', 'n_votes', 'n_comments', 'total_engagement', 'popular', 'num_words', 'avg_
word_length', 'sentiment']
((venv) ) pavanipasupuleti@Pavanis-MacBook-Air codingproblem %
```

- Logistic Regression with undersampling achieved the highest accuracy (~71.5%) and AUC (~0.68) but had very low sensitivity, meaning it poorly detected popular reviews.
- XGBoost showed slightly lower accuracy (~70.8%) with marginally better sensitivity, but overall performance remained similar across feature subsets.
- The Neural Network had the lowest accuracy (~68.6%) but the highest sensitivity, indicating better detection of popular cases at the cost of more false positives.

```
((venv) ) pavanipasupuleti@Pavanis-MacBook-Air codingproblem % python src/predict_review.py


Enter the review text.
Press ENTER on an empty line to finish.

Worst fantasy novel I've read in years.
The writing is lazy, the characters are forgettable, and the story goes nowhere.
I forced myself to finish it and completely regret the time I wasted.
If you enjoyed this book, I genuinely want to know why.


------------------------------
Prediction: POPULAR
------------------------------
```

The user entered a strongly negative book review describing poor writing and wasted time. Despite the negative sentiment, the model predicted the review as POPULAR.

```
((venv) ) pavanipasupuleti@Pavanis-MacBook-Air codingproblem % python src/predict_review.py


Enter the review text.
Press ENTER on an empty line to finish.

I enjoyed this book.
The characters were nice and the story was easy to follow.
It was a pleasant read and I would recommend it to fans of the genre.


------------------------------
Prediction: NOT POPULAR
------------------------------
```

The user entered a positive review expressing enjoyment and recommendation of the book. The model predicted the review as NOT POPULAR, showing that positive sentiment alone does not guarantee popularity.

5. Conclusions and Challenges Faced

Conclusion

- Text-based features combined with TF-IDF representations are effective for predicting review popularity.
- Logistic Regression performs competitively while being simple and interpretable.
- Sentiment and review length significantly influence popularity prediction.
- The trained model can successfully predict popularity for unseen reviews.

Challenges Faced

- Class imbalance: Popular reviews are fewer compared to non-popular ones, requiring under sampling.
- Large text data: TF-IDF feature extraction increases dimensionality and computation cost.
- File path issues on Windows required careful folder structure management.
- Model persistence required explicit creation of directories before saving models.

Despite these challenges, the system successfully predicts review popularity with reasonable accuracy and serves as a practical application of text-based machine learning.